Academic Year: 2026-27 Sap Id: 60003240292

Shri Vile Parle Kelavani Mandal'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Empowered Autonomous) Affiliated to the University of Mumbai

Department of Information Technology

COURSE CODE: DJS23ICPC501 **DATE: 16/08/2026**

COURSE NAME: Computer Networks **CLASS: T. Y. B.Tech(I3 – 1)**

Experiment No. 4

CO/LO: Analyse various concerns at different layers in reference models using suitable techniques.

Aim: Write a program to implement Hamming code for error detection and correction

EXERCISE

```
import random

def hamming(data):
    m = len(data)
    r = 0
    while 2 ** r < m + r + 1:
        r += 1
    n = m + r
    code = [0] * (n + 1)
    j = 0
    for pos in range(1, n + 1):
        if pos & (pos - 1) == 0:
            continue
        code[pos] = int(data[j])
        j += 1
    pos = 1
    while pos <= n:
        parity = 0
        for i in range(1, n + 1):
            if i & pos:
                parity ^= code[i]
        code[pos] = parity
        pos *= 2
    result = ""
    for pos in range(n, 0, -1):
        result += str(code[pos])
    return result

def decode(received):
    n = len(received)
```

```
    code = [0] * (n + 1)
    for pos in range(1, n + 1):
        code[pos] = int(received[n - pos])
    error_position = 0
    pos = 1
    while pos <= n:
        parity = 0
        for i in range(1, n + 1):
            if i & pos:
                parity ^= code[i]
        if parity:
            error_position += pos
        pos *= 2
    return received, error_position

def correct(received, error_position):
    if error_position == 0:
        return received
    bits = list(received)
    index = len(bits) - error_position
    bits[index] = "1" if bits[index] == "0" else "0"
    return "".join(bits)

def process(data, frame_size):
    binary = ""
    for byte in data:
        binary += format(byte, "08b")
    padding = (frame_size - len(binary) % frame_size) % frame_size
    binary += "0" * padding
    total_blocks = len(binary) // frame_size
    error_blocks = max(1, total_blocks // 20)
    selected_blocks = random.sample(range(total_blocks), error_blocks)
    errors = 0
    corrections = 0
    for block in range(total_blocks):
        start = block * frame_size
        end = start + frame_size
        data_block = binary[start:end]
        encoded = hamming(data_block)
        received = encoded
        if block in selected_blocks:
            bits = list(encoded)
            error_index = random.randint(0, len(bits) - 1)
            bits[error_index] = "1" if bits[error_index] == "0" else "0"
            received = "".join(bits)
            errors += 1
        received, error_position = decode(received)
        corrected = correct(received, error_position)
        if error_position != 0:
            corrections += 1
        if block < 5:
```

```
            print("Corrected:", corrected)
    print("Corrected all")
    return total_blocks, errors, corrections

def main():
    filename = "abcd.pdf"
    with open(filename, "rb") as file:
        data = file.read()
    result16 = process(data, 16)
    result32 = process(data, 32)
    print("Frame Size | Total Blocks | Blocks with Error")
    print("16         |", result16[0], "|", result16[1])
    print("32         |", result32[0], "|", result32[1])
    print("Corrections for 16-bit:", result16[2])
    print("Corrections for 32-bit:", result32[2])

if __name__ == "__main__":
    main()
```

```
PS D:\Python\CN> & C:\Python314\python.exe d:/Python/CN/exp4.py
Corrected: 0000110101010101011
Corrected: 0110000100010000011010
Corrected: 1000101000101100100010
Corrected: 1110101000111001000010
Corrected: 0101000000101110000001
Corrected all
Corrected: 0110000100010001000001001010101010101001
Corrected: 1110111000111010010001010010110010001 1
Corrected: 1010110011010010001010000010111000000 0
Corrected: 1011001001010111011010101101001100101
Corrected: 0000110000000001001000111000101000000 10
Corrected all
Frame Size | Total Blocks | Blocks with Error
16         | 8382 | 419
32         | 4191 | 209
Corrections for 16-bit: 419
Corrections for 32-bit: 209
```

Observation:

Sr. No	Frame Size	Total Blocks	Blocks with Error
1	16	8382	419
2	32	4191	209

Conclusion:

Hamming Code has been implemented successfully for detecting and correcting errors in the transmitted data. It enhances the efficiency of data communication by introducing redundancy and detecting the position of a bit that is incorrect.

Reference:

[1] www.geeksforgeeks.org